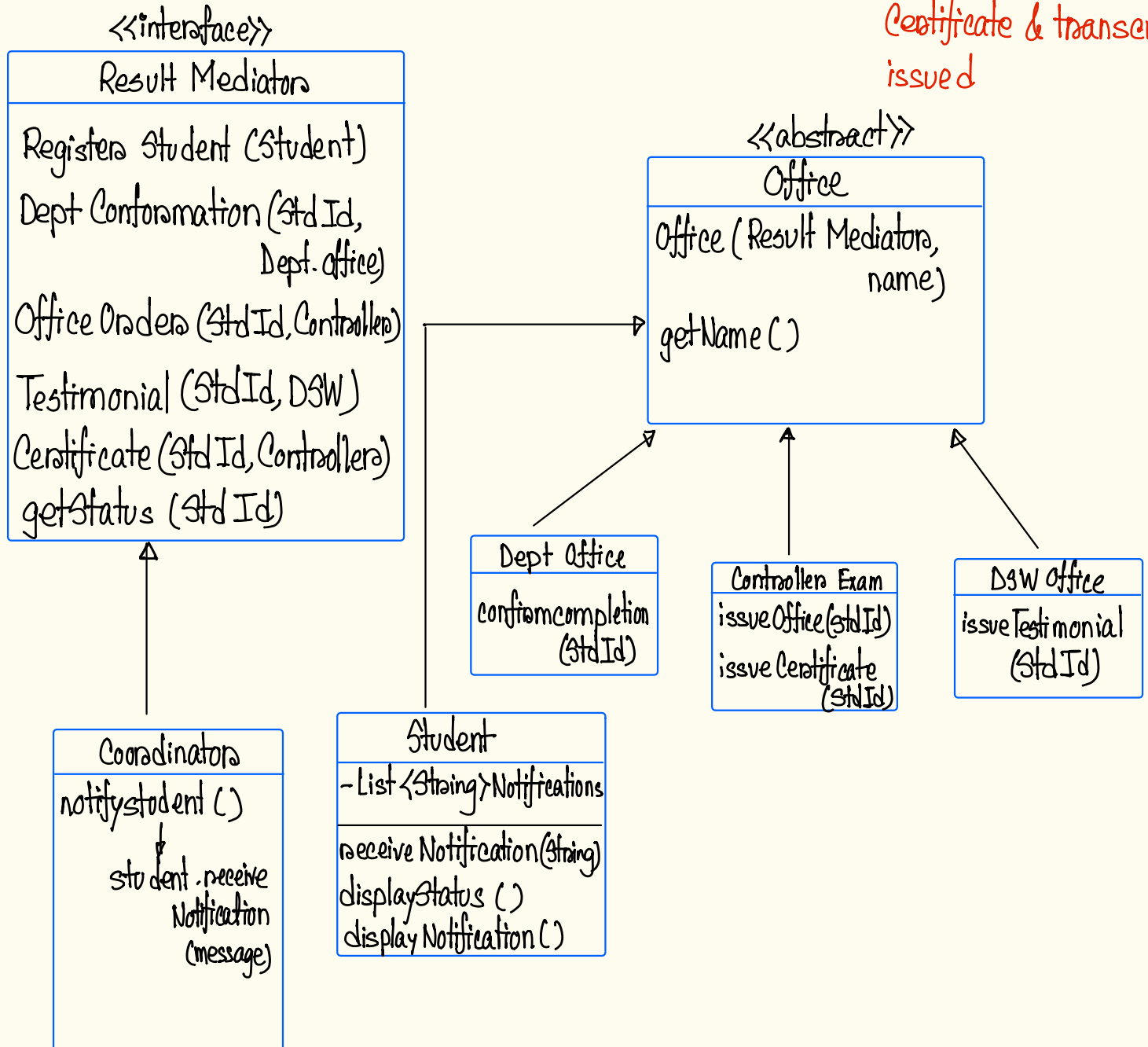


Offline 3

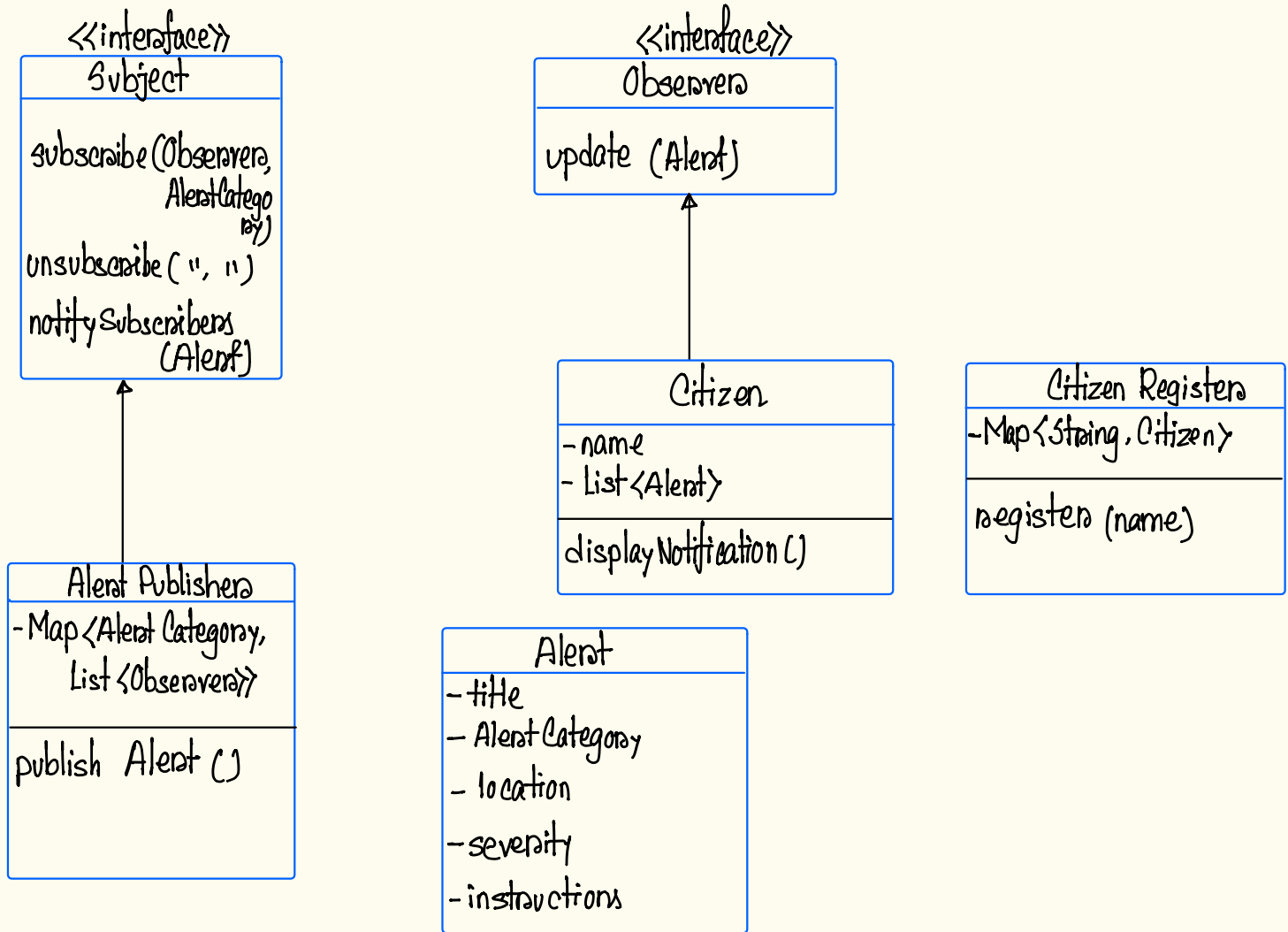
Task-2: Mediator

1. Department Office → ensure that student has completed academic requirements
2. Controllers of Examination → Gets notify by dept office. Now issues office orders for result publication
3. DSW → Gets notify by controllers. issued testimonial
↳ DSW does the work. Certificate & transcript issued
4. Student



Task-1: Observer

Alert → Earthquake, floods & fires



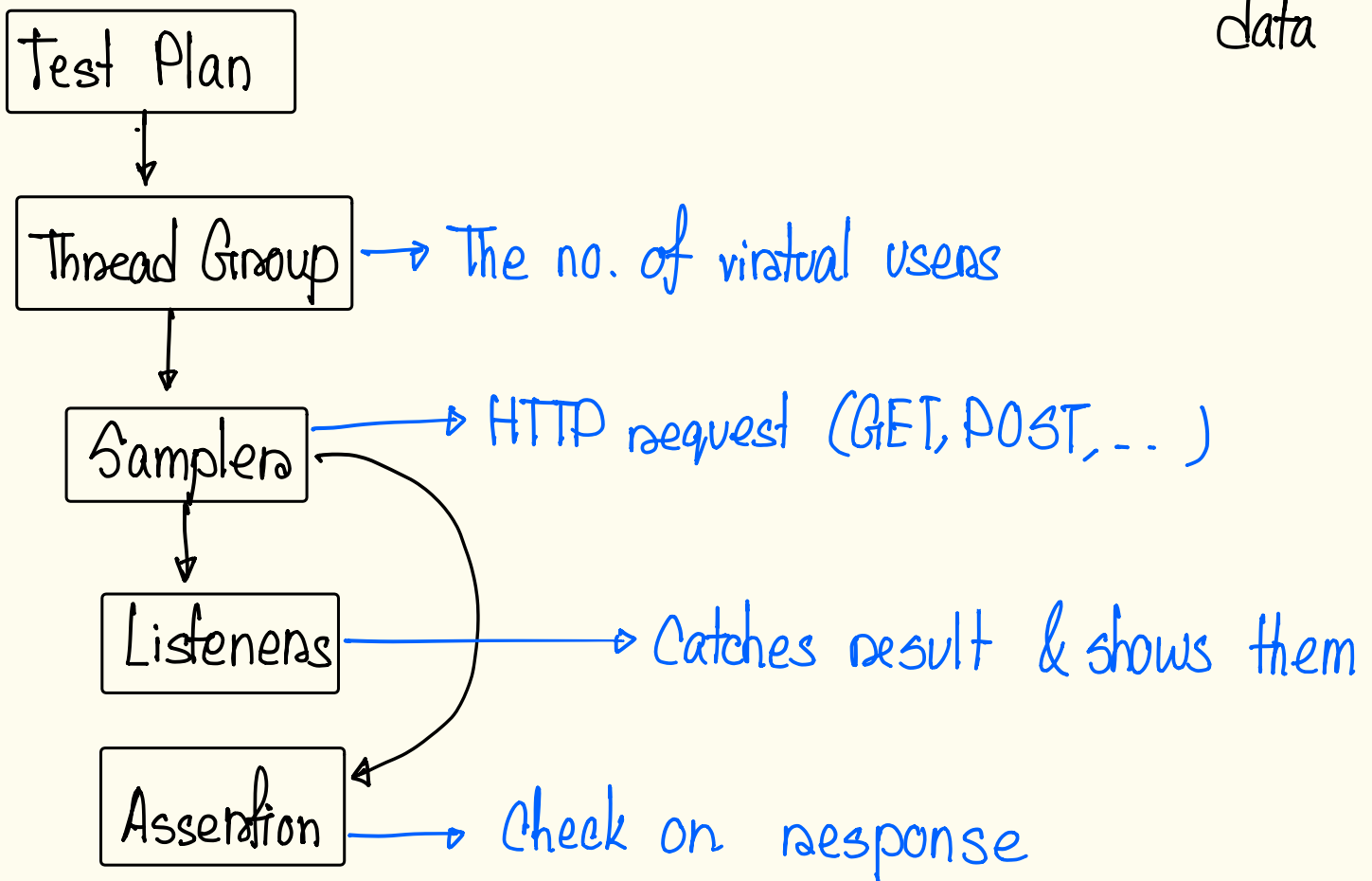
Load Testing

Performance → behaviour during maximum Load

Jmeters → Many virtual users & hit the servers

→ It speaks with protocol (HTTP)

Test Plan → 1. How many virtual users
2. Which requests, in which order, with which data



How to run the test?

1. Test Plan → Add → Threads (Users) → Thread Group
2. Thread Group → Add → Samplers → HTTP Request
3. Configure the request → protocol, server, path
4. Thread Group → Add → Listeners → View Results Tree

5. Add View Results in Table

6. Save.

Request

Protocol $\rightarrow$ how to speak (<https>)

Server $\rightarrow$ who to speak (www.google.com)

Path $\rightarrow$ which endpoint to that server ([/login](#))

Parameters $\rightarrow$ key-value pairs the server uses to decide what to send back

Thread Groups

No. of threads $\rightarrow$ how many virtual users

Ramp-up period $\rightarrow$ The time needed to start all the threads.

Loop Count

1. Amount of work

2. Total time varies with response time

Duration

1. Amount of time

2. Enable the scheduler and set Durations in second

Connect Time $\rightarrow$ Time to connect the server

Latency $\rightarrow$ Sending first byte to server

Sample Time/Elapse $\rightarrow$ sending last byte to server

Connect Time < Latency < Elapse

1. Thread Group → Add → Logic Controllers → Recording controllers
 2. Test Plan → Add → Non-Test Elements → HTTP(s) Test Script Recorder
 3. Start the Recorder once
 4. Settings → Privacy & Security → Security → Manage Certificates
↓
Import ← Authorities
 5. Point your browser's proxy at localhost:8888
 6. Start the recorder, browse the site & end it
- *Certificate first, proxy second
- *After you record, undo the browser's proxy

But how does browsing become load testing?

A fair question: if the recorder just captured one Chrome window doing one journey, how do we get *load*?

Three moves:

- **Record once.** You, one browser, one journey ⇒ a list of samplers in the Recording Controller.
- **Multiply with the Thread Group.** Threads=10, Loops=5 tells JMeter to replay that same list **50 times**, spread across **10 parallel workers**.
- **Isolate with the Cookie Manager.** Each thread gets its own cookie jar ⇒ ten independent logins, not ten requests sharing one session.



CLI

The GUI is friendly. It is also:

- a memory hog (Your listeners keep everything in RAM),
- slower than the CLI on the same test plan,
- incapable of feeding results into a CI/CD pipeline, and
- officially not recommended for load runs (JMeter itself tells you so at startup).

The workflow is:

Build the .jmx in the GUI → Run it from the CLI → Open the results back in the GUI (or as HTML).

1

Confusing "the GUI is for building" and "the CLI is for running" is a rite of passage. You will do it once, watch a test plan crawl, and never do it again.

Process

1. Test Plan

2. Test → Add → Config → HTTP Request default (Configuration)

↳ Protocol: http

IP: 103.94.135.91

Port: 8080

3. Test → Add → Config → HTTP Cookie (for login)

4. Test → Thread (For low) → User: 50

Ramp: 100

5. Test → Add → Non-Test elements → Script Recorder

6. Thread-low → Logic Controllers → Recording Controller (x5)

1. Home

2. Notice

3. Courses

4. Login

5. Bandwidth

7. Start → Stop → Firefox → manage certificate → import

8. Show Proxy

9. Start → Target Controller → Now for every recording controller

10. Remove Unwanted record

11. Test → Add → Listener → ① View Results Tree

② Summary Report

③ Aggregate

④ View Results in Table

12. 2 parameters in login →

13. Test → Add → Assertion → Duration Assertion → 250ms

14. Run the load test (through CLI)